Prueba de desempeño - Bases de datos

"La información no es conocimiento. La única fuente de conocimiento es la experiencia organizando el caos."

1. Introducción

Te has unido al equipo de ingeniería de **LogiTech Solutions**, una consultora especializada en modernización de sistemas para retail y comercio electrónico.

Uno de sus clientes más grandes, el gigante de suministros "**MegaStore Global"**, enfrenta una crisis operativa. Durante años, han manejado todo su inventario, ventas, proveedores y clientes en **un único archivo maestro de Excel.**

El volumen de datos ha crecido tanto que el archivo es inmanejable: hay inconsistencias en los precios, direcciones de clientes duplicadas con errores ortográficos y es imposible saber el stock real en tiempo real.

Tu misión: Actuar como Arquitecto de Datos y Desarrollador Backend para migrar este "sistema legado" a una arquitectura moderna, escalable y persistente, exponiendo la información a través de una API REST.

2. Objetivos

El objetivo de esta prueba es evaluar tu capacidad para:

1. **Analizar y diseñar:** Tomar un set de datos plano y desorganizado, y proponer una arquitectura de persistencia adecuada.
2. **Arquitectura del modelo:** Diseñar un esquema de base de datos que elimine redundancias innecesarias y asegure la integridad de la información.
3. **Persistencia:** Implementar un motor relacional **SQL** y un motor **NoSQL**.
4. **Desarrollo backend:** Construir una API con **Express.js** para la gestión de datos.

5. **Inteligencia de negocio:** Resolver requerimientos de información complejos mediante consultas o agregaciones.

6. **Log de auditorías:** Manejar log de transacciones en escenarios especificos desde Mongo DB.

3. Requisitos técnicos

Fase 1: Análisis y modelado de datos

- **Fuente de datos:** Analizarás la estructura de un archivo plano (Excel/CSV) que contiene información mezclada:
    - *ID Transacción, Fecha, Nombre Cliente, Email Cliente, Dirección, Categoría Producto, SKU, Nombre Producto, Precio Unitario, Cantidad, Nombre Proveedor, Contacto Proveedor.*
- **Diseño del modelo:**
    - Debes definir qué información será almacenada en un motor relacional (PostgreSQL o MySQL) y cuál en un motor NoSQL (MongoDB).
    - Debes justificar técnicamente tu decisión.
    - No se evaluará únicamente que ambos motores estén implementados, sino la coherencia de tu arquitectura.
    - **Para SQL:** Debes aplicar las formas normales (1FN, 2FN, 3FN) según se requiera, para descomponer la información en tablas relacionadas.
    - **Para NoSQL (ej. MongoDB):** Debes diseñar un esquema eficiente, justificando qué datos se normalizan (referencias) y qué datos se incrustan (embedding) para optimizar la lectura.
- **Diagrama:** Debes entregar una representación visual de tu modelo (DER y Diagrama de Colecciones) hecho en una herramienta externa (draw.io o similar).

Fase 2: Implementación de base de datos (db_megastore_exam)

- Es obligatorio utilizar MongoDB (NOSQL) y un motor relacional SQL (PostgreSQL o MySQL) dentro de la solución propuesta.
- **Convención de Nombres:** Todas las tablas/colecciones y atributos/campos deben estar en **Inglés**.
- **Integridad:**
    - **(SQL):** Uso estricto de Llaves Primarias (PK), Foráneas (FK) y Restricciones (UNIQUE, NOT NULL).
    - **(NoSQL):** Uso de Validación de Esquema (Schema Validation) o índices únicos para evitar duplicidad de entidades clave.

Fase 3: Migración masiva de datos (El reto)

- El sistema debe ser capaz de ingerir el archivo plano desorganizado y distribuir la información en tu nuevo modelo de datos.
- **Idempotencia:** Debes implementar una lógica que evite la duplicación de entidades maestras.
    - *Ejemplo:* Si el cliente "Juan Pérez" aparece en 10 filas de transacciones diferentes en el CSV, tu base de datos solo debe crear **un** registro de cliente y vincular las 10 órdenes a él.
- Este proceso de carga debe ser ejecutable mediante un script o un endpoint específico de la API.

Fase 4: Backend CRUD (Express.js)

- Desarrolla un **servidor RESTful** usando Node.js y Express.
- Selecciona **UNA entidad principal** de tu modelo (ej: *Products*, *Orders* o *Suppliers*) y desarrolla un **CRUD completo** para gestionarla, además de guardar un log de registros en MongoDB cada que se elimine, para poder hacer auditoria sobre ellos.

- El código debe ser modular, limpio y organizado.
- La conexión a la base de datos debe ser robusta y manejar errores correctamente.

Fase 5: Consultas avanzadas (Business Intelligence)

El Gerente de operaciones necesita visualizar la siguiente información a través de Postman:

- **Análisis de proveedores:**
  - *"Necesito saber qué proveedores nos han vendido más productos (en cantidad de items) y cuál es el valor total del inventario que tenemos asociado a cada uno."*
- **Comportamiento del cliente:**
  - *"Quiero ver el historial de compras de un cliente específico, detallando productos, fechas y el total gastado en cada transacción."*
- **Productos estrella:**
  - *"Genera un listado de los productos más vendidos dentro de una categoría específica, ordenados por ingresos generados."*

(Nota: Para SQL, esto requiere JOINs/Group By. Para NoSQL, esto requiere Aggregation Framework).

4. Entregables

1. **Repositorio de GitHub:**
   a. Código fuente del proyecto.
   b. Carpeta /docs con:
      i. Diagrama del Modelo de Datos (Imagen o PDF).
      ii. Archivo CSV de muestra (La "Data Cruda" que usaste para las pruebas).
      iii. Scripts de Base de Datos (DDL para SQL y Scripts de Validación para NoSQL).

c. **Colección de Postman** (exportada como .json).

d. README.md.

5. Documentación (README.md)

Debe estar redactado en **Inglés** (para simular un entorno internacional) e incluir:

- Pasos para desplegar y ejecutar el proyecto localmente.
- **Justificación del modelo:** Explica *por qué* estructuraste la base de datos de esa manera.
    - *(Crítico para NoSQL: ¿Por qué embebiste vs referenciaste?)*
    - *(Crítico para SQL: Explica el proceso de normalización).*
- Guía de cómo usar el endpoint/script de Migración Masiva.
- Descripción de los Endpoints disponibles.

7. Criterios de aceptación

- **Lógica de migración:** El algoritmo de carga detecta y gestiona registros existentes correctamente (previene duplicados).
- **Modelo de datos:**
    - (SQL) Cumple con la 3FN.
    - (NoSQL) Demuestra un balance entre eficiencia de lectura/escritura (no es un dump plano).
- **Calidad del backend:** Código organizado, uso de variables de entorno (.env), códigos de estado HTTP correctos.
- **Consultas:** Responden con exactitud a las preguntas de negocio planteadas.

8. Es un plus si:

- Utilizas procedimientos almacenados, triggers o vistas.
- Documentas instalación de motores en Ubuntu con comandos.
- Si entregas una interfaz visual para un cliente final.